

Whats the difference between asp.net and asp.net core ?

The main difference is that **ASP.NET** is an older, legacy framework tied strictly to Windows, while **ASP.NET Core** is a modern, rebuilt framework that is cross-platform, faster, and open-source.

Comparison Overview

Feature	ASP.NET (Legacy)	ASP.NET Core (Modern)
Operating System	Windows only	Cross-platform (**Windows, Linux, macOS**)
Open Source	No (Proprietary Microsoft)	Yes (**Open-source** on GitHub)
Performance	Standard	**High-performance** (significantly faster)
Architecture	Monolithic & heavy	Modular (**Lightweight** using NuGet packages)
Project Setup	`Global.asax`, `Web.config`	`Program.cs`, `appsettings.json`
Supported Technologies	Web Forms, MVC, Web API, SignalR	MVC, Razor Pages, Web API, SignalR, **Blazor**
Development Status	Maintenance mode (no new features)	Actively developed by Microsoft and the community

Key Differences Explained

- **Cross-Platform Flexibility:** ASP.NET apps require a Windows server running IIS to host. ASP.NET Core can be developed and hosted anywhere, including **Linux containers and Docker images**, saving massive infrastructure costs.
- **Performance Boost:** ASP.NET Core was entirely rewritten from scratch to be lean and fast. In standard industry benchmarks, ASP.NET Core minimal APIs can process over **2 million requests per second**, vastly outperforming older versions and matching or beating other modern backend frameworks.
- **No More Web Forms:** If you rely on old `aspx` Web Forms, you cannot migrate them natively to Core. ASP.NET Core replaces this paradigm with newer frontend models like **Blazor** or Razor Pages.
- **Modern Architecture:** ASP.NET Core features built-in **Dependency Injection (DI)**, uniform integration of MVC and Web API frameworks, and a cloud-ready configuration system.

Learning **.NET MAUI Blazor Hybrid** using **.NET 10** and the upcoming **.NET 11** is an excellent choice. **Razor components** instead of XAML. [1, 2, 3, 4]

Here is what you need to know about learning and working with .NET MAUI Blazor Hybrid across these two versions.

🚀 The Version Landscape: .NET 10 vs. .NET 11

The current stable release is **.NET 10 (LTS)**, while **.NET 11 (STS)** is in its Release Candidate phase. [1, 2]

Feature / Aspect	.NET 10 (Current LTS)	.NET 11 (Upcoming STS)
Support Type	**Long Term Support (LTS)** until Nov 2028	**Standard Term Support (STS)** until Nov 2028
C# Version	C# 14	C# 15
Core Architecture	Standard multi-targeting	**CoreCLR runtime** replaces Mono for Android workloads (massive performance gain)
Stability Focus	Enterprise-grade, production-safe	Bleeding edge, rapid feature updates

---

💎 Key Features to Learn in .NET 10 & 11

When diving into your learning journey, make sure to focus on these powerful capabilities introduced in these versions:

1. The Shared "Hybrid & Web App" Template

- **What it is:** Introduced to streamline cross-platform architecture, this template sets up a **Blazor Web App** and a **.NET MAUI Blazor Hybrid app** that share 100% of their UI inside a single **Razor Class Library (RCL)**.

- **Why it matters:** As noted by developers building real-world apps, debugging directly in a mobile emulator can be slow. With this template, you can **run and debug your code entirely in a standard desktop web browser first**, then deploy the exact same UI straight to iOS or Android. [1, 2, 3]

2. **Intercepting Web Requests**

- .NET 10 added the `WebResourceRequested` event to the `BlazorWebView`. You can now natively intercept, modify headers, redirect, or inject custom local streams for internal app requests directly from your C# backend. [1]

3. **Advanced JavaScript Interop**

- The underlying `HybridWebView` features new asynchronous initialization events (`WebViewInitializing` and `WebViewInitialized`). This makes it incredibly easy to configure native, platform-specific webview properties (like configuring WebView2 on Windows or Chromium on Android) right before your Blazor app paints the screen. [1, 2]

🛠️ How to Set Up Your Learning Environment

To begin practicing with the absolute latest tools, update your environment:

1. Download and install **Visual Studio 2022** (or VS Code with the C# Dev Kit).
2. Open your terminal or command prompt and ensure your workloads are fully up to date:
   bash
   ```
   dotnet workload install maui
   ```
   Use code with caution.
3. Create your first unified hybrid project using the CLI: [1, 2, 3]
   bash
   ```
   dotnet new maui-blazor-web -o MyFirstHybridApp
   ```
   Use code with caution.

💡 Core Concepts to Master First

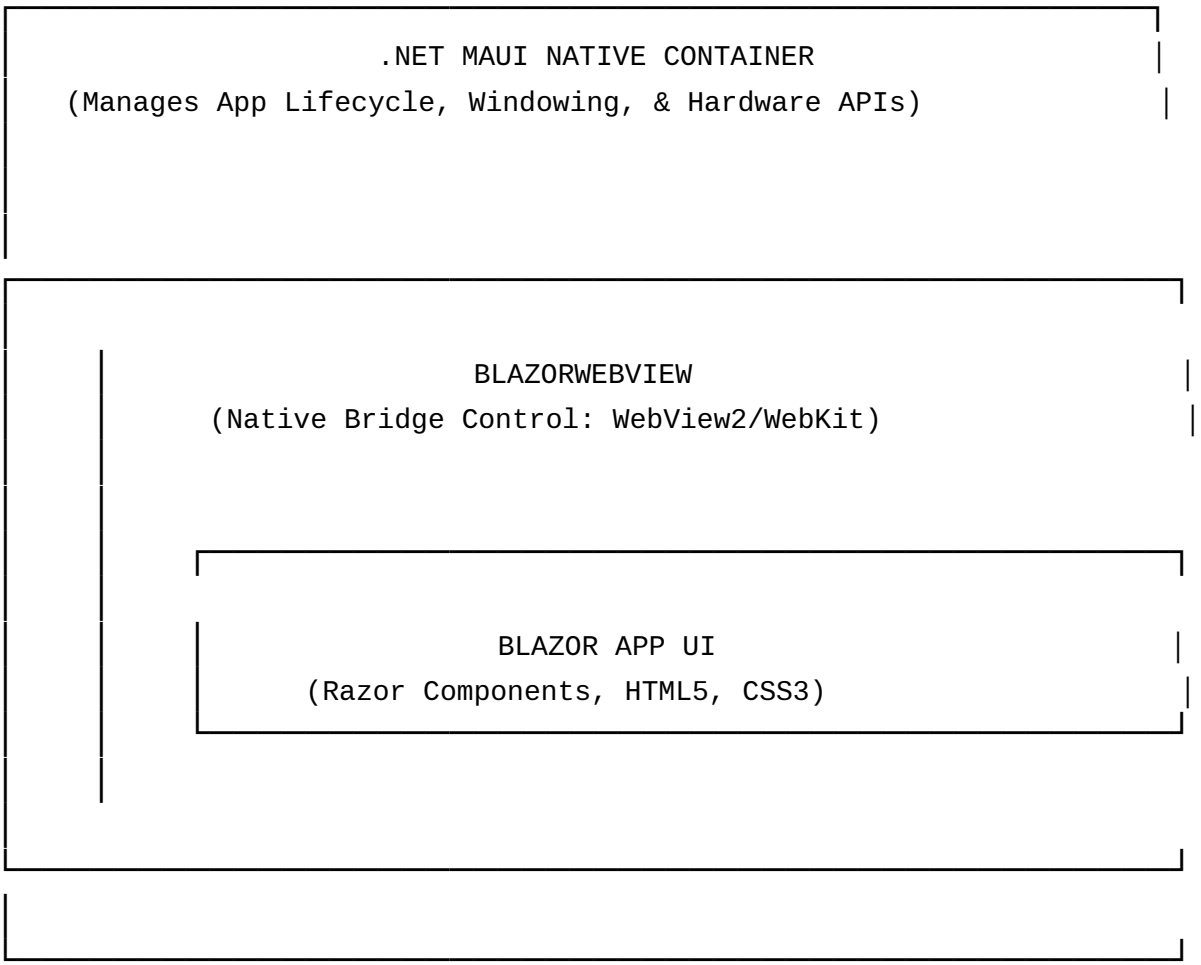
- **Native Interop / Dependency Injection:** Learn how to use a single C# interface to fetch data on the web, but dynamically swap it to pull hardware sensor data (like GPS or Camera) when running natively inside the MAUI container. [1, 2]
- **CSS & Platform Styling:** Because Blazor Hybrid renders using the native device engine (WebView2 on Windows, Safari on iOS, Chromium on Android), you will want to master tailoring your CSS to gracefully adapt to different screen dimensions and platform rendering behaviors. [1, 2]

Cont…

- **Fast Focus: Building a Blazor MAUI Hybrid App**
  Build hybrid apps with .NET MAUI

  The architecture of a **.NET MAUI Blazor Hybrid app** blends native container technology with web-based rendering. [1]
  Instead of compiling your C# and HTML into native OS platform controls (like traditional .NET MAUI does with XAML), a hybrid app **runs your web UI directly inside a native hardware wrapper**. [1, 2, 3]

🧱 The 3 Pillars of Hybrid Architecture



1. The Native Container (.NET MAUI)

Your app is fundamentally a native operating system application (`.apk` for Android, `.app` for iOS/macOS, `.exe` for Windows). MAUI handles the underlying system requirements: [1, 2]

- App startup and lifecycle events
- Native multi-window management
- Direct access to the **MAUI Essentials API** for hardware access (GPS, Bluetooth, Biometrics, Secure Storage) [1, 2, 3, 4]

2. The Native Bridge (`BlazorWebView`)

This is the core architectural magic. Instead of utilizing a remote browser web server, MAUI embeds a specialized native control called `BlazorWebView`. [1, 2]

- On **Windows**, this leverages Microsoft's native **WebView2** runtime.
- On **iOS/macOS**, it leverages Apple's native **WKWebView**.
- On **Android**, it leverages **Chromium**. [1, 2]

Unlike standard web apps, there is **no network latency or HTTP layer** required to display the UI. Your Blazor code executes locally inside the device's native .NET process, and the `BlazorWebView` maps the output directly to the device engine. [1, 2]

3. The UI Layer (Blazor Razor Components)

Inside that native bridge sits your web code. You construct your interface entirely with HTML tags, standard CSS frameworks (like **Tailwind CSS** or **Bootstrap**), and **Razor components**. [1, 2, 3]

🔄 JavaScript Interop in .NET 10 & 11

Because the app runs inside a native web view wrapper, communication between the native device layer (C#) and the web canvas (JavaScript) is incredibly fast. [1]

Starting in **.NET 10**, Microsoft upgraded the underlying hybrid pipeline to auto-forward unhandled JavaScript errors natively into standard **.NET Exceptions**. You can easily pass messages back and forth using the global JavaScript interop layer: [1]

csharp

```
// Inside a Razor Component (.razor)
await JSRuntime.InvokeVoidAsync("showNativeNotification", "Hello from Blazor!");
```

Use code with caution.

💎 Hybrid vs. Standard Web / Progressive Web Apps (PWAs)

- **No Browser Sandbox:** A standard web app or PWA is sandboxed by the browser for security—it cannot access raw local files, custom system directories, or specific hardware elements. Hybrid apps have **100% unrestricted access** to the device because they run as an installed native execution package. [1, 2, 3]
- **True Local Processing:** The HTML/CSS assets are compiled directly into the application payload. The app works completely **offline by default** without needing service workers to cache page files. [1, 2]

Difference Between .NET MAUI and Blazor Hybrid and When ...
NET MAUI provides direct and full access to device features like: Camera. GPS. Sensors. it usually requires integration with nativ...

Choosing Between .NET MAUI and Blazor Hybrid for Your ...
NET MAUI uses XAML and native UI components, ensuring optimized rendering and responsiveness. Blazor Hybrid, in contrast, renders ...

It is completely natural to look forward to it, especially since **.NET 11** brings some of the biggest architectural upgrades to mobile development since the framework launched! The most exciting change coming in **.NET 11** is that Microsoft is replacing the older Mono runtime with the highly optimized **CoreCLR runtime for Android workloads**. For you as a developer, this means a massive leap in raw application performance, faster startup times, and significantly lower memory usage on mobile devices.

What to Focus on Right Now

While we wait for the final release of .NET 11, you can confidently start learning on **.NET 10** today. Because .NET 11 is built directly on top of the foundations of .NET 10, **100% of the Blazor Hybrid code, Razor components, and CSS layout skills you practice right now will translate perfectly**.

To help you get ready, here is a quick visual checklist of how your learning path should look over the coming weeks:

- **Phase 1 (The Core):** Master creating standard **Razor Components** and managing application state.
- **Phase 2 (The Layout):** Pick a modern CSS framework like **Tailwind CSS** or **Bootstrap** to handle responsive UI designs that look great on both phone screens and desktop monitors.
- **Phase 3 (The Hybrid Connection):** Learn how to use the **Dependency Injection (DI)** container to pass native device data smoothly into your web views.

1. The Official Microsoft Learn Quick-Start

- **Best for:** Getting the development environment configured properly in 15 minutes.
- **What it covers:** Step-by-step setup guides, managing your workload installation, and understanding how the `unset BlazorWebView` projects structure works natively.
- **Resource Link:** **Build a mobile and desktop app with Blazor Hybrid and .NET MAUI** [1, 2, 3]

2. The Official Microsoft .NET Presentations Workshop (GitHub)

- **Best for:** Practical, hands-on learning through building a single, fully functioning application.
- **What it covers:** This is an outstanding hands-on repository maintained by the .NET community team. You build a "Monkey Finder" app from scratch. It teaches you how to pull live JSON data, use native device GPS/location sensors, mix native map wrappers with Blazor code, and theme the app.
- **Resource Link:** **Official .NET Blazor Hybrid Workshop Repository** [1, 2]

3. Highly Rated "Quick Study" Books

If you learn best by following structured, cohesive project blueprints rather than fragmented articles, these specific guides target the modern .NET ecosystem: [1]

- **For Concepts & Architecture:** **_.NET MAUI Blazor Hybrid Made Easy_** covers everything from component structures to building out local SQLite databases. [1]
- **For Absolute Beginners:** ***Blazor Hybrid and .NET MAUI for Beginners*** maps out the exact configurations required for .NET 10 tools and guides you through building a practical task application. [1]

💡 Pro-Tip for Fast Learning

When utilizing these materials, always create your project using the **MAUI Blazor Hybrid and Web App** template: [1]

bash

```
dotnet new maui-blazor-web -o MyApp
```

Use code with caution.

Instead of launching a slow Android or iOS mobile emulator every time you change a line of UI code, **run the Web project first**. You can debug, test, and style your Razor components instantly inside your desktop web browser. Because the code lives in a shared Razor Class Library (RCL), it will run perfectly when you switch targets to mobile later. [1, 2, 3]

Researched by Randy Kirchin all code and this summary are parts of the web not written by me.